

When Is a Self-Improving Agent’s Diagnosis Worth Verifying? Allocating Evidence That Costs a Substantial Fraction of an Episode

Anonymous ACL submission

Abstract

Self-improving LLM agents let an analyst model read failed trajectories, blame a component, and patch it. Because attributions are unreliable, recent systems verify a patch before applying it: they replay the failure with the patch, ask an LLM judge, or re-run tasks that used to pass. Methods for this step usually assume that some evidence is cheap. We measured it on three tool-use settings, a fault-injected customer-service environment and the retail and airline domains of τ^2 -bench, using over 18,000 agent runs and judge checks. Every kind of evidence consumes a substantial fraction of an episode: a single-step judge check costs 23–72% of a full replay, and its sensitivity is only 0.37–0.57. We therefore treat verification as allocating a budget of episodes among targeted replays, null replays that measure spurious recovery, and regression runs. A value-of-information decomposition splits what verification can be worth into avoiding harm, choosing among candidate patches and discovering effective edges the analyst missed, and bounds the first by a patch’s expected harm. On τ^2 -bench, a task-matched unpatched baseline places a patch’s mean effect on previously solved tasks at -0.003 (retail) and -0.006 (airline), with 90% intervals of about ± 0.03 , so the average patch neither helps nor harms other tasks. Which verification policy is best is still not identified by our intervention banks: posterior values span several oracles, because patches differ in harm by about 0.04 and adjudication is unstable. On held-out tasks, applying the attributed patches without evidence raised success from 0.76 to 0.88 on retail and from 0.68 to 0.75 on airline, and no verified patch set did better than it on either domain; ablations attribute most of the retail gain to a single step-limit patch. We release the environments, fault suites and intervention banks.

1 Introduction

A growing family of systems improves LLM agents from their own failures: the agent runs, an analyst model reads the failed trajectories, and its diagnosis drives a change to a prompt, a tool description or the orchestration (Shinn et al., 2023; Zhao et al., 2024; Hu et al., 2025; Zhang et al., 2025a; Agrawal et al., 2026; Chen et al., 2026). The diagnosis step, *failure attribution*, is unreliable (Zhang et al., 2025b; Liu et al., 2026): models label failures by their visible symptom rather than their cause, so a loop that trusts them will often change the wrong component. The common remedy is to verify before acting: patch the blamed component and replay the failure (Ma et al., 2025; Lin et al., 2026; Shah, 2026), ask a judge whether the failing step is fixed, or re-run tasks that used to pass and keep the patch only if nothing breaks (Chen et al., 2026).

Verification costs agent episodes, so the natural research question is how to spend them. A tempting answer, and the one we started from, treats LLM judgements as cheap evidence and full replays as expensive, and allocates a budget across both by Bayesian experimental design. In a simulator built on these assumptions that approach needs half the budget of the strongest baseline (Appendix H). On real agents the premise fails. Across three tool-use settings, a judge check costs 23–72% of a full replay, because the judge must read the trajectory, and it detects only 37–57% of real repairs. No evidence is cheap: every kind consumes a substantial fraction of an episode.

We therefore pose the problem with every piece of evidence priced in episodes, and ask a prior question: *when is a diagnosis worth verifying at all?* Our contributions are:

- A formulation of failure-driven self-improvement as *episode-priced evidence allocation*. A patch’s net effect on success decomposes into repair on its target category,

spurious recovery that mimics repair, and regressions on other tasks; targeted replays, null replays and regression runs each estimate one term (§3).

- A value-of-information decomposition: what any verification can add to acting on the attributions splits into avoiding harm, which is bounded by the expected harm of the default patch, choosing among candidate patches, and discovering causes the analyst missed. The familiar bound $\min(\pi G, (1 - \pi)H)$ is the special case in which only the edge is uncertain. With the sample sizes needed to estimate each term, this tells practitioners where verification could pay before they spend a budget on it (§4).
- Measurements on ShopDesk and offline intervention banks on τ^2 -bench retail and airline, with paired replays and judge checks for every patch, null replays, and 20 regression runs for every patch an original policy accepts. Evaluating eleven policies under several cost models, with independent adjudication and with alternative harm scorings, shows that the ranking of policies hinges on a patch’s effect on previously solved tasks. A task-matched unpatched baseline puts the mean of that effect within ± 0.03 of zero, but per-patch differences still leave the ranking open, and on held-out tasks of both domains no improvement from verification over applying the attributed patches was detected (§7).
- A net-effect allocation method that models the three terms with per-category spurious recovery and chooses experiments by knowledge gradient per unit cost. It is a reference allocation built from the measurements, not a demonstrated advance: it is among the better verifying policies under several scorings, but no policy, ours included, is best everywhere (§5, §7). CARVE, the multi-fidelity design we started from, is reported as a design study in Appendix H.

2 Related Work

Failure attribution for LLM agents. Zhang et al. (2025b) introduced the Who&When benchmark and compared all-at-once, step-by-step and binary-search prompting for locating the responsible agent and step; Liu et al. (2026) extended it to

longer and multimodal traces and found that attribution remains far from solved. AgenTracer uses counterfactual replay and fault injection to build training data for a small attribution model (Zhang et al., 2026); AgentDebug pairs an error taxonomy with targeted corrective feedback (Zhu et al., 2025); ErrorProbe traces dependencies backwards and keeps a verified memory of error patterns (Li et al., 2026). Taxonomies such as MAST (Cemri et al., 2025) give attribution a fixed vocabulary, and AdaMAST induces a system-specific taxonomy from traces whose axes map to intervention points (Cemri et al., 2026). AdaMAST validates its taxonomy by agreement between LLM annotators; we instead treat the mapping from categories to components as a causal claim to be checked by intervention.

Verification by intervention. DoVer generates hypotheses from logs and validates each by editing a message or plan and re-running, with several runs per intervention (Ma et al., 2025). REFLECT tests diagnosis-specific patches by targeted replay and feeds verified outcome changes back into attribution (Lin et al., 2026). CausalFlow scores steps by their causal responsibility under counterfactual interventions (Bonagiri et al., 2026), and Causal Agent Replay formalises step interventions in a structural causal model with a point-of-commitment rule (Shah, 2026). AgentDebugX provides a toolkit with policy-gated reruns and a store of failure–diagnosis–repair bundles (Zhu et al., 2026). All of these operate per trajectory or per hypothesis and do not decide how to allocate a shared budget across hypotheses, patches and levels of fidelity, which is our focus. Our method can use any of them as the replay primitive.

Self-improving agents. Reflexion, Self-Refine and ExpeL turn feedback into verbal lessons (Shinn et al., 2023; Madaan et al., 2023; Zhao et al., 2024); prompt optimisers such as ProTeGi, OPRO, TextGrad and DSPy use failures or textual feedback to rewrite prompts (Pryzant et al., 2023; Yang et al., 2024; Yuksekgonul et al., 2025; Khattab et al., 2024); ADAS, the Darwin Gödel Machine and GEPA search over agent designs or prompts guided by evaluation and failure logs (Hu et al., 2025; Zhang et al., 2025a; Agrawal et al., 2026). Harness-Fix attributes failures to harness artefacts, repairs them, and promotes a repaired harness only if it solves more validation tasks while breaking at most two (Chen et al., 2026); its validation is regression-

aware, but it uses whole-task runs on fixed splits rather than allocating evidence per patch. Bayesian-Agent keeps Beta posteriors over the reliability of skills and applies threshold rules to patch, split or retire them (Wu et al., 2026); its posterior is over skills rather than causal edges, and actions are chosen by fixed thresholds rather than by the value of information. CARVE is complementary: it decides which proposed fixes to trust before they are applied.

Experimental design. Our selection rule is the KG(*) variant of the knowledge gradient (Frazier et al., 2008; Frazier and Powell, 2010), within Bayesian experimental design (Chaloner and Verdinelli, 1995). Related problems include multi-fidelity bandits and optimisation (Kandasamy et al., 2016; Poloczek et al., 2017), thresholding and best-arm identification (Locatelli et al., 2016; Audibert et al., 2010; Russo, 2016), sequential testing (Wald, 1945), active causal structure learning (Tong and Koller, 2001; Hauser and Bühlmann, 2014) and causal bandits (Lattimore et al., 2016). In software engineering, delta debugging isolates failure-inducing changes by experiment (Zeller and Hildebrandt, 2002), and program-repair work warns that a patch passing a failing test may not fix the fault (Smith et al., 2015), which our separation of edges from patch quality reflects. We use the knowledge gradient as an allocation rule, but our main question is a prior one from decision analysis: when the information is worth its cost at all.

3 Problem: Episode-Priced Evidence

Setting. An agent is assembled from *components* j (prompt sections, tool descriptions, configuration). On a task distribution it fails with rate F ; an LLM groups its failures into categories i with shares f_i . For each *cell* (i, j) that the analyst blames, it proposes K candidate patches k that rewrite component j . Let $e_{ij} \in \{0, 1\}$ indicate whether changing j can repair category i at all, and q_k the fraction of category- i failures that patch k repairs if so. Separating e from q matters because a patch that fails to help may be aimed at the wrong component or merely be a bad patch. We call a cell with $e_{ij} = 1$ an *effective edge*. This is an interventional notion: some change to j repairs failures of category i . It need not mean that j was the original cause of those failures, and our ground truth (§6) establishes only that a tested patch helps.

Net effect. Applying patch k of cell (i, j) changes the success rate by

$$\Delta_k = F f_i e_{ij} q_k - (1 - F) \rho_k - c_a - c_w (1 - e_{ij}), \quad (1)$$

where ρ_k is the patch’s absolute excess failure rate on tasks that used to succeed (regressions, which may hit any category): such a task fails a re-run with probability r_0 unpatched and $r_0 + \rho_k$ patched, so $\rho_k < 0$ is possible, $c_a \geq 0$ is a cost charged for every applied change (review, maintenance) and $c_w \geq 0$ a penalty charged only for a change aimed at a wrong component (risk not captured by ρ_k). We report costs as fractions of the failure mass F and vary both. The goal is to choose a set of patches, at most one per cell, that maximises $\sum_k \Delta_k$ on held-out tasks.¹

Evidence. Three kinds of experiment, each costing a fraction of an episode, inform Eq. 1:

- A *targeted replay* resumes a category- i failure from the attributed step with the patch applied; it succeeds with probability $b_i + (1 - b_i) e_{ij} q_k$.
- A *null replay* does the same without a patch and succeeds with probability b_i , the category’s *spurious recovery* rate. Without it, replay success cannot be interpreted.
- A *regression run* executes a previously successful task from scratch with the patch and fails with probability $r_0 + \rho_k$, where r_0 is the natural re-run failure rate.

A fourth, a *single-step check* that regenerates only the failing step and asks an LLM judge whether the error is gone, is often proposed as the cheap option. We measure its cost and accuracy in §7. Unlike classic multi-fidelity settings, all four cost a substantial fraction of an episode, within a factor of four of each other.

4 When Can Verification Pay?

Before any evidence, the net effects $\Delta = (\Delta_k)$ of the candidate patches are uncertain, with a prior that reflects the analyst’s accuracy and what is known about patch quality and regressions. The *default* applies, in each cell, the patch with the largest $\mathbb{E}[\Delta_k]$ if that is positive. Verification observes outcomes Y and then chooses; its value is $V(Y) = \mathbb{E}[\max_S \mathbb{E}[\sum_{k \in S} \Delta_k \mid Y]] - \max_S \mathbb{E}[\sum_{k \in S} \Delta_k]$,

¹Effects of patches to different components are treated as additive. Held-out success of whole patch sets and a leave-one-out ablation (§7, Appendix E) are the only checks.

before the cost of the experiments. Write $x^+ = \max(x, 0)$ and $x^- = \max(-x, 0)$.

Proposition 1 (Value of verification). *For any experiments Y , $V(Y)$ is at most the value of learning Δ exactly, which is the sum over cells of*

$$V_{ij} = \mathbb{E}[\max_k \Delta_k^+] - \max_k (\mathbb{E}\Delta_k)^+. \quad (2)$$

With k^* the default patch of the cell,

$$V_{ij} \leq \underbrace{\mathbb{E}[\max_k \Delta_k^+ - \Delta_{k^*}^+]}_{\text{choosing among patches}} + \underbrace{\mathbb{E}[\Delta_{k^*}^-]}_{\text{avoiding harm}}, \quad (3)$$

with equality when the default applies k^* , and $\mathbb{E}[\Delta_{k^*}^-] \leq (1 - F) \mathbb{E}[\rho_{k^*}^+] + c_a + c_w P(e_{ij}=0)$.

Corollary 1 (Edge-only verification). *Suppose a cell has a single candidate patch whose net effect depends only on e_{ij} : $\Delta = G$ if $e_{ij} = 1$ and $\Delta = -H$ if $e_{ij} = 0$, with $G, H \geq 0$ known (so q_k and ρ_k are known given the edge). With $\pi = P(e_{ij}=1)$, $V_{ij} = \min(\pi G, (1 - \pi)H)$.*

Proofs are in Appendix A. The proposition separates three sources of value. *Avoiding harm*: in a cell the default would patch, verification can save at most the patch’s expected downside, which needs regressions or change costs; replay success alone cannot create it. *Choosing among patches*: if candidate patches differ in quality, finding the best one has value even when nothing is harmful. *Discovery*: in a cell the default leaves alone, the whole term $\mathbb{E}[\max_k \Delta_k^+]$ is value from finding an edge the analyst missed. The corollary is the special case in which only the edge is uncertain; it does not hold when quality or regressions are uncertain given the edge (for example, $\pi = 1$ with $\Delta = +0.10$ or -0.05 equally likely gives $V = 0.025$). We measure all three terms on our banks in §7, using the measured net effects in place of the unknown Δ .

Proposition 2 (Cost of evidence). *Under Wald’s approximations, a sequential test that decides e_{ij} from targeted replays with error δ needs on average about $\log \frac{1-\delta}{\delta} / \text{KL}(b + (1-b)q \parallel b)$ replays when the edge is real. Detecting a regression of size ρ against a base rate r_0 with a one-sided level- α test and power $1 - \beta$ needs about $(z_\alpha + z_\beta)^2 r_0 (1 - r_0) / \rho^2$ regression runs.*

Two consequences follow. First, targeted replays are uninformative about the harm term, because they only see the target category: a patch can pass every replay and still break other tasks. Second, small regressions are expensive to see. With $r_0 =$

0.1, a regression of $\rho = 0.25$ is detected with 90% power by 20 runs, whereas $\rho = 0.03$ needs about 850. Since many small regressions add up, failing to detect them is not evidence that they are absent; §7 therefore reports interval estimates rather than significance counts.

Three questions. Proposition 1 bounds what information is worth *before* paying for it; it is the standard value-of-information argument (Chaloner and Verdinelli, 1995), and what is new here is its decomposition for patches and its use on measured banks. Two further questions are separate. *Allocation*: given a fixed budget of episodes, which experiments to run (§5); our policies and banks address this one. *Whether to buy verification at all*: a patch, once applied, changes success on every future episode, so if the agent will run N more episodes, an experiment costing C episodes is worth buying when $N V(Y) > C$ in success units, and a knowledge-gradient policy should stop when N times its best gain per unit cost falls below one. On our banks the oracle’s whole repairable gain is 0.07–0.11 in success per episode, so a policy that buys 40 episodes of evidence and thereby recovers a further 10% of it pays for itself after 3,500–6,000 future episodes. Our experiments fix the budget and do not choose N ; the horizon decides how much of the value in Proposition 1 is worth paying for.

5 Allocation Policies

When verification is warranted, the budget must still be allocated across cells, patches and experiment types. We compare eleven policies.

Net-effect allocation (ours). We place a spike-and-slab model on each cell. The edge prior $P(e_{ij}=1) = \pi_{ij}$ comes from the analyst’s attribution counts, shrunk towards a base rate by its audited accuracy (Eq. 6 in Appendix H). Each q_k has a grid posterior from a Beta(2, 2) prior, each b_i a Beta posterior updated by null replays, and each ρ_k a Beta posterior updated by regression runs. Its estimate of Eq. 1 is $P(e_{ij}=1 \mid \mathcal{D}) F f_i \mathbb{E}[q_k \mid \mathcal{D}] - (1 - F) \max(0, \mathbb{E}[r_k \mid \mathcal{D}] - r_0) / (1 - r_0) - c$, where r_k is the patch’s failure rate in regression runs. Two choices make it conservative: it charges c on every change it applies, and it scales the regression term by $1/(1 - r_0) \approx 1.1$. It scores cells independently and does not enforce one patch per component; the deployed set resolves such conflicts

(§7). The policy repeatedly runs the experiment (targeted replay of a patch, null replay of a category, or regression run of a patch) that maximises the n -step knowledge gradient of the decision value divided by its measured cost, with $n \in \{1, 2, 4, 8\}$ (Frazier et al., 2008), and finally applies every patch whose posterior net effect is positive. Unlike CARVE, it has no single-step checks, a separate b_i per category, and a regression term.

Baselines. *Apply attributed* uses no evidence: for each category it applies the first patch of the most-blamed component and of any component with at least 40% of the blame. *Replay-each* tests hypotheses in order of blame with three replays and accepts on two successes, as in do-then-verify methods (Ma et al., 2025). *HarnessFix-style, per patch* adds two regression runs that must break nothing; these counts are ours. *HarnessFix-style, bundle gate* is closer to the original, which validates a whole bundle of repairs on held-out tasks and promotes it if it solves at least one more task and breaks at most two (Chen et al., 2026). We apply the *Apply attributed* bundle, draw validation episodes from the bank, and allow three attempts. *Uncertainty sampling* replays the cell with the largest frequency-weighted posterior uncertainty, with or without three cheap checks per patch first. CARVE is our multi-fidelity design method (Appendix H); we also run it with only full replays, with the judge’s measured accuracy, and with a quality-dependent false-positive rate.

6 Environments and Intervention Banks

Environments. *ShopDesk* is a customer-service environment we built, with a seeded database, six deterministic tools, exactly checkable answers and 11 named agent components. τ^2 -*bench retail and airline* (Yao et al., 2024; Barres et al., 2025) are dual-control customer-service domains with an LLM user simulator and database-state rewards. We split each domain policy into sections and add every tool description and the step limit as components, giving 24 components for retail and 21 for airline. The agent is DeepSeek-V4-Flash. Attribution, taxonomy induction (Wan et al., 2024), patch generation and judging use DeepSeek-V4-Pro.

Faults that look like defects. Each training episode runs clean or with one injected fault, written as a plausible authoring mistake rather than an obvious artefact. In retail: a policy section allow-

ing gift-card refunds, a cancellation rule without the reason check, a product-tool description that confuses item and product ids, and a tight step limit. In airline: a wrong bag allowance, a 7-day instead of 24-hour cancellation window, a missing basic-economy restriction, proactive compensation, and a tight step limit. Clean episodes contribute natural failures: 22 of 137 on retail and 28 of 112 on airline.

Banks. For each τ^2 domain we take 36 cells, the attributed cells in order of blame plus random unattributed cells of populated categories, and generate three patches per cell. For every patch we record 20 paired targeted replays and single-step checks. We also record 20 null replays per category and 4 regression runs per patch, extended to 20 for the 121 patches that any of the original policies accepted. Policies run offline by drawing recorded outcomes from the bank, with replacement, with costs in measured tokens and full replay = 1, over 200 seeds; a rerun that draws without replacement is a sensitivity check. Resampling seeds measure policy variability given the bank, not new evidence about the patches. *Adjudication*: a cell is an effective edge if its best patch’s replay success exceeds the category’s null rate by at least 0.2 (one-sided Fisher $p < 0.05$), giving 4 effective edges on retail and 7 on airline. Choosing the best of three patches on the same replays makes this label optimistic, so we also adjudicate on held-back halves of the bank and bootstrap it. *Baseline*: we ran the unpatched agent 10 times on each of the 111 regression tasks (1,110 runs), so that each patch’s regression runs can be compared with the unpatched failure rate of the same tasks; these runs were collected about 12 hours after the regression runs, and unpatched held-out success did not change over that time (0.816 vs. 0.817 on clean retail episodes). *Harm*: our primary scoring is the posterior expected value of each policy’s patch set, with every patch’s ρ_k , its excess failure rate over its task-matched baseline, drawn from a hierarchical posterior (patches share a normal population whose mean μ and spread τ are estimated, given a resample of the baseline runs) and edges and repair rates from a bootstrap of the replays; we report 90% intervals. Charging only significant regressions (one-sided binomial $p < 0.05$) is a sensitivity analysis. Values are fractions of the *repair-only oracle*, the failure mass repaired by the effective edges’ best patches with no cost, a fixed yardstick for every scoring. Inde-

Budget (episodes)	τ^2 retail (development)						τ^2 airline (held out)					
	posterior, $c=0$			posterior, $c_w=0.02$			posterior, $c=0$			posterior, $c_w=0.02$		
	10	40	80	10	40	80	10	40	80	10	40	80
Apply attributed	0.64	0.64	0.64	0.08	0.08	0.08	0.99	0.99	0.99	0.60	0.60	0.60
Replay-each	0.74	0.92	0.89	0.22	0.59	0.63	0.78	0.63	0.63	0.49	0.45	0.48
HarnessFix-style, per patch	0.37	0.74	0.99	0.37	0.66	0.80	0.16	0.45	0.62	0.15	0.41	0.51
HarnessFix-style, bundle	0.39	0.65	0.86	0.12	0.18	0.36	0.58	0.90	0.97	0.34	0.54	0.58
Uncertainty sampling	0.98	1.12	1.03	0.62	0.90	0.80	0.87	0.88	0.89	0.64	0.68	0.70
Uncertainty + checks	1.21	1.11	1.13	0.88	0.94	0.97	0.83	0.72	0.72	0.61	0.54	0.58
CARVE, replay only	0.76	1.14	1.21	0.35	0.79	0.79	1.01	0.98	1.06	0.70	0.67	0.75
CARVE	1.15	0.83	0.71	0.88	0.72	0.67	0.80	0.48	0.31	0.57	0.36	0.25
CARVE, measured check	0.76	1.14	1.21	0.35	0.79	0.79	0.98	0.97	1.07	0.68	0.67	0.77
CARVE, robust check	1.01	1.46	1.33	0.59	1.08	0.91	0.96	0.98	1.06	0.66	0.67	0.76
Net-effect (ours)	0.66	0.89	1.10	0.52	0.78	0.96	0.66	0.71	0.79	0.53	0.61	0.68

Table 1: Posterior expected value of each policy’s patch set on the τ^2 intervention banks, as a fraction of the repair-only oracle; bold is the best per column. Each of 300 draws takes every patch’s excess failure rate over its task-matched unpatched baseline from the hierarchical posterior, the baseline from a resample of its runs, and edges and repair rates from a bootstrap of the replays, and averages over 40 resampling seeds; right blocks also charge 0.02 of the failure mass per patch aimed at a wrong edge. Values above 1 arise when patches are estimated to reduce failures on previously solved tasks. Intervals are in Table 4, and the same table with only significant regressions charged is Table 6.

pendently of the bank, we apply the accepted patch sets to 40 unseen retail tasks and 20 unseen airline tasks.

Protocol. Retail was the development domain for every method, including the variants in Appendix F. Airline was evaluated once for the original policies and once, later, for the net-effect method, which was frozen on retail beforehand.

7 Results

7.1 What evidence costs and what it shows

Table 2 reports the quantities that Propositions 1 and 2 depend on.

No evidence is cheap. A judge check costs 23–72% of a full replay, because the judge reads the trajectory, and it recognises only 37–57% of real repairs. Its information per unit cost is below that of a replay in every environment: the break-even cost ratio is about 0.1, and the measured ratio is 0.23 or more. Its false-positive rate is no higher for bad patches than for good ones, so a judge fooled by plausible bad patches did not occur. The judge is weak, not misled.

Replays need controls. Without any patch, 20–23% of replays from the failing step succeed anyway, and the rate ranges from 0 to 0.5 across categories. A replay success rate of 0.5 is strong evidence in one category and none in another. Deciding one edge takes 5–7 replays even with the right

b_i .

Harm is zero on average but not measurable per patch. Against the task-matched unpatched baseline, 5 of 121 patches are significantly harmful and 9 helpful on previously solved tasks (one-sided 95%), about what chance would produce. Pooled over all patches, the excess failure rate is -0.003 on retail and -0.006 on airline, with 90% intervals of ± 0.03 – 0.04 ; wrong-edge patches alone give $+0.004$ and -0.007 . With r_0 from paired training re-runs instead, the estimates were $+0.03$ and -0.03 with intervals twice as wide; on retail that r_0 was biased low (0.09 vs. 0.14 on the same tasks). Individual patches remain unmeasured: only 3–6 patches per domain are shown to lie within ± 0.10 (TOST), harms above 0.10 are excluded for about a quarter, and on retail the hierarchical model estimates a spread of $\tau \approx 0.04$ between patches. An excess of 0.04 is about 0.1 of the failure mass, comparable to what an effective edge repairs, and measuring it per patch would take hundreds of runs (Proposition 2).

7.2 Where could verification add value?

Table 3 evaluates Proposition 1 on the banks. Charging only significant regressions, the default reaches 0.84 (retail) and 0.60 (airline) of the oracle, avoiding harm is worth nothing, and the remaining value lies in discovering edges the analyst did not blame (0.35, 0.74) and choosing better patches (0.06, 0.01). Under the posterior, the default’s ex-

	ShopDesk	Retail	Airline
Analyst accuracy $\hat{r}$	0.72	0.41	0.29
Effective edges among blamed cells	–	0.18	0.33
Judge check: cost / replay	0.72	0.28	0.23
sensitivity	0.57	0.37	0.41
false-positive rate	0.21	0.11	0.13
break-even cost ratio	0.08–0.13	0.09	0.10
Spurious recovery b (mean)	–	0.23	0.20
range over categories	–	0–0.50	0–0.50
Repair q of effective edges (median)	–	0.50	0.41
Replays to decide an edge	–	5	7
Re-run failure r_0 , paired trials	–	0.11	0.14
unpatched, regression tasks	–	0.14	0.12
Patches harmful / helpful (95%)	–	2 / 5	3 / 4
worse than peers, same tasks	–	0/61	2/60
$\rho > 0.10$ excluded (95%)	–	18/61	16/60
Within ± 0.10 (TOST)	–	3/61	6/60
Pooled ρ , all patches	–	–0.003	–0.006
90% interval	–	[–0.04, 0.04]	[–0.04, 0.02]
Mean μ (hierarchical)	–	–0.003	–0.006
90% interval	–	[–0.03, 0.02]	[–0.03, 0.02]
Spread τ of ρ_k	–	0.04	0.00

Table 2: Measured quantities. “Effective edges among blamed cells” is the precision of applying every attributed patch. The break-even cost ratio is the cost below which judge checks are more informative per unit cost than replays (Appendix H). Regression rows use the 121 patches with 20 regression runs, each compared with 10 unpatched runs of every task its regression runs used (task-matched baseline); “worse than peers” compares patches with each other on the same tasks. The pooled interval bootstraps tasks and both sets of runs.

pected value is 0.64 of the oracle on retail and 0.99 on airline, with intervals from about -3 to $+4$ and -1 to $+3$: the mean harm μ is near zero, but each applied patch carries its own uncertain ρ_k . The value of perfect information is then several oracles on retail, most of it from resolving each patch’s effect on previously solved tasks: avoiding harm (1.7), choosing the patch with the best ρ_k (2.8), and finding unblamed patches whose ρ_k may be negative (the large discovery term). On airline, where the model finds no spread between patches ($\tau \approx 0$), avoiding harm is worth 0.19 and choice 0.13. Only regression runs, at the sample sizes of Proposition 2, provide this value; it is zero only under the significance scoring.

7.3 Policies on the banks

Posterior expected value. Table 1 reports each policy’s posterior expected value. With μ near zero, the ranking is decided by which patches each policy happens to apply. On retail, CARVE with a robust check (1.46 at budget 40) and uncertainty sampling with checks (1.21 at 10) lead and applying everything scores 0.64; verifying policies beat it in 41–78% of draws, and with $c_w = 0.02$ applying everything falls to 0.08. On airline, applying everything is best at $c = 0$ (0.99, the single best

	Scoring	Default	Harm	Choice	Discovery
Retail	posterior, $c = 0$	0.64	1.72	2.76	10.93
	posterior, $c_w = 0.02$	0.08	2.04	2.57	9.71
	significant only, $c = 0$	0.84	0.00	0.06	0.35
Airline	posterior, $c = 0$	0.99	0.19	0.13	2.65
	posterior, $c_w = 0.02$	0.60	0.37	0.13	2.12
	significant only, $c = 0$	0.60	0.00	0.01	0.74

Table 3: Value of perfect verification over the no-evidence default (apply the attributed patches), split by the terms of Proposition 1, as fractions of the repair-only oracle. *Posterior*: expectations over the hierarchical harm posterior and the replay bootstrap (§6). *Significant only*: measured net effects with only significant regressions charged. Terms are per cell and additive.

policy in 23–47% of draws), CARVE variants and the bundle gate come close, and with $c_w = 0.02$ uncertainty sampling and CARVE variants lead (0.68–0.77). Every interval spans more than one oracle (Table 4), so the policies are statistically indistinguishable: what remains uncertain is each patch’s own ρ_k and the adjudication of edges.

Robustness. Table 4 rescores the budget-40 decisions. (i) *Adjudication is unstable.* On held-back halves only 1–3 of retail’s 4 effective edges and 3–7 of airline’s 7 are confirmed, and bootstrapping the replays yields anywhere from 2 to 19 (retail) and 4 to 19 (airline) edges. Almost every policy’s value falls under split adjudication, most for applying everything. (ii) *Retail’s value is mostly the step limit.* Without the step-limit component, every retail policy falls below zero, while airline keeps 83% of its oracle and HarnessFix-style and CARVE stay on top. (iii) *Deployment constraints matter.* Reducing each set to one patch per component, as deployed, raises applying everything from 0.16 to 0.47 on retail and from 0.15 to 0.46 on airline, because overlapping wrong cells stop being charged twice. (iv) Drawing outcomes without replacement moves retail by up to 0.32 and airline by at most 0.05.

7.4 Held-out tasks

Table 5 applies each policy’s patch set to the agent on unseen tasks, which tests whole sets, interactions included, with runs independent of the bank. On retail, applying the attributed patches raises success from 0.76 to 0.88 ($+0.12 \pm 0.04$, paired, task-clustered); on airline, from 0.68 to 0.75 ($+0.07 \pm 0.06$). No improvement from verification was detected on either domain: every verified set differs from applying everything by -0.04 to 0.00 overall. This is not an equivalence result.

Policy (budget 40)		Posterior, $c_w=0.02$		Sensitivity: significant harm only						
		Value [90%]	$P(> \text{apply})$	c_w	c_a	Split	Natural	No step limit	One per comp.	No repl.
Retail	Apply attributed	0.08 [−3.38, 3.32]	–	0.16	0.01	−0.46	−1.25	−0.61	0.47	0.16
	HarnessFix-style, per patch	0.66 [−0.07, 1.46]	0.61	0.42	0.34	0.12	−0.82	−0.27	0.42	0.54
	HarnessFix-style, bundle	0.18 [−2.24, 2.48]	0.53	−0.31	−0.42	−0.80	−1.99	−0.83	−0.08	−0.45
	Uncertainty sampling	0.90 [−0.86, 2.79]	0.74	0.23	0.09	−0.13	−1.10	−0.64	0.29	0.28
	CARVE	0.72 [−0.26, 1.77]	0.66	0.19	0.07	0.04	−0.89	−0.25	0.20	0.51
	Net-effect (ours)	0.78 [−0.01, 1.65]	0.65	0.41	0.32	0.23	−0.86	−0.35	0.41	0.52
Airline	Apply attributed	0.60 [−1.15, 2.52]	–	0.15	−0.08	−0.41	−0.07	−0.03	0.46	0.15
	HarnessFix-style, per patch	0.41 [−0.03, 0.87]	0.43	0.37	0.25	0.17	0.28	0.20	0.38	0.38
	HarnessFix-style, bundle	0.54 [−1.06, 2.33]	0.27	0.12	−0.09	−0.30	−0.09	−0.04	0.39	0.10
	Uncertainty sampling	0.68 [−0.74, 2.03]	0.59	0.25	0.01	0.04	0.05	0.05	0.41	0.26
	CARVE	0.36 [−0.36, 1.12]	0.36	0.23	0.08	−0.03	0.24	0.22	0.25	0.23
	Net-effect (ours)	0.61 [−0.19, 1.36]	0.52	0.31	0.14	0.17	0.23	0.13	0.35	0.26

Table 4: Uncertainty and robustness at budget 40 (fractions of the repair-only oracle). *Posterior*: mean and 90% interval over the hierarchical harm posterior and the replay bootstrap, and the probability of beating Apply attributed in the same draw. Sensitivity columns charge significant regressions and $c_w = 0.02$ per wrong change unless stated: c_a , 0.02 per applied change instead; *Split*, policies see a random half of every outcome list and adjudication uses the other half (5 splits); *Natural*, values reweighted to the category shares of natural failures; *No step limit*, the step-limit component removed from the bank; *One per comp.*, the accepted set reduced to one patch per component as deployed; *No repl.*, policies draw bank outcomes without replacement.

Patch set	Retail			Airline	
	All	Clean	− Apply [90%]	All	− Apply [90%]
Unpatched	0.76	0.82	–	0.68	–
Apply attributed	0.88	0.87	–	0.75	–
Replay-each	0.83	0.87	−0.04 [−0.10, 0.02]	–	–
HarnessFix-style, per patch	0.83	0.90	−0.04 [−0.13, 0.04]	0.71	−0.04 [−0.12, 0.04]
HarnessFix-style, bundle	0.85	0.87	−0.03 [−0.05, −0.00]	0.72	−0.03 [−0.09, 0.03]
Uncertainty sampling	0.86	0.86	−0.02 [−0.07, 0.04]	–	–
Uncertainty + checks	0.85	0.82	−0.03 [−0.08, 0.03]	–	–
CARVE, replay only	0.88	0.87	0.00 [−0.05, 0.06]	–	–
CARVE	0.84	0.81	−0.04 [−0.10, 0.03]	0.74	−0.01 [−0.09, 0.07]
CARVE, measured check	0.85	0.86	−0.03 [−0.08, 0.02]	–	–
CARVE, robust check	0.86	0.84	−0.01 [−0.06, 0.04]	–	–
Net-effect (ours)	0.83	0.83	−0.04 [−0.10, 0.01]	0.75	−0.00 [−0.09, 0.08]

Table 5: Held-out success, budget 10, mean over the patch sets of 5 seeds. Retail: 40 unseen tasks $\times$ 3 episodes (38 clean, 82 fault-injected, 12 with the step-limit fault). Airline: 20 unseen tasks $\times$ 3 episodes (25 clean), five policies. “− Apply” is the paired difference from Apply attributed over all episodes, with a task-clustered 90% interval.

With a non-inferiority margin of 0.05 in success, chosen after the fact, applying everything is non-inferior to 8 of the 10 verified sets on retail and to the two HarnessFix-style sets on airline, but not to CARVE or the net-effect method there, whose intervals reach +0.08.

What the gain consists of. Ablations on retail (Appendix E) show that the step-limit patch helps only on step-limit-fault episodes (+0.25 to +0.58), the other attributed patches add +0.09 [0.01, 0.18] elsewhere, no single patch’s removal is detectable, and on 158 clean episodes applying everything and the verified sets are indistinguishable.

8 Discussion

What the baseline settled and what it did not. Per-patch regression runs cannot settle the mean effect μ of a patch on solved tasks: with $r_0 \approx 0.1$ they detect only harms of 0.25 or more, while $\mu = 0.03$ summed over ten patches cancels the gain of the effective edges. 1,110 task-matched unpatched runs narrowed μ to within ± 0.03 of zero, which removed the shift that had decided the earlier ranking. Rankings now depend on per-patch effects and adjudication, which the banks cannot resolve. Held-out tests of whole sets detected no gain from verification and excluded large joint harm on both domains. This favours bundle-level validation, as in HarnessFix, for the harm term, and per-patch evidence for discovery and patch choice, where the value of information remains.

Recommendations. (i) Price evidence in measured tokens. (ii) Run null replays per category. (iii) Measure harm against a task-matched unpatched baseline and score it by its posterior, not by significance tests. (iv) Adjudicate on held-back data. (v) Set the budget from the deployment horizon (§4).

9 Conclusion

On real agents, no verification evidence is cheap. On our τ^2 banks the average patch neither helps nor harms solved tasks, yet policy rankings remain unidentified, and on held-out tasks no gain from verification was detected.

Limitations

We study one agent model and one analyst model on three customer-service settings. The banks have 36 cells per domain, and their 4 and 7 effective edges are only partly confirmed on held-back replays; our edges are effective interventions, not established causes. Regression estimates rest on 20 runs per patch; the unpatched baseline (10 runs per task) was collected about 12 hours after them, and only held-out success checks for drift between the two; the hierarchical posterior assumes normally distributed patch effects. Effects are treated as additive within the bank; whole sets are tested on held-out tasks at budget 10 only, with 60 episodes per set on airline, and the leave-one-out runs are too noisy to measure interactions. Most of retail’s held-out gain on fault-injected episodes comes from the step-limit fix. Most failures come from injected faults written to resemble real defects (natural failures are 16–25%), and reweighting to natural failures is not a bank built from them. We evaluate a single round of improvement under fixed budgets, not a multi-round loop or a deployment horizon. The net-effect method was developed on retail after we had seen airline results for the other policies, although not its own. Replay from a failing step requires deterministic tools.

Ethics Statement

Self-improving agents change their own behaviour without a person reviewing each change. That no improvement from verification was detected on our held-out tasks, and that the average patch did not harm other tasks, does not show that verification is unnecessary, and our measurements cannot exclude small harms per patch; it should not be read as licence to skip review. On ShopDesk a single patch silently broke a quarter of tasks (Appendix G). Our environments use synthetic customer data only.

References

Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2026. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations*.

Jean-Yves Audibert, Sébastien Bubeck, and Rémi

Munos. 2010. Best arm identification in multi-armed bandits. In *Proceedings of the 23rd Conference on Learning Theory*.

Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. 2025. τ^2 -bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*.

Akash Bonagiri, Devang Borkar, Gerard Janno Anderias, Setareh Rafatirad, and Houman Homayoun. 2026. CausalFlow: Causal attribution and counterfactual repair for LLM agent failures. *arXiv preprint arXiv:2605.25338*.

Mert Cemri, Andrei Cojocaru, Melissa Pan, Shu Liu, Shubham Agarwal, Alexander Krentsel, Jay Tang, Kannan Ramchandran, Joseph E. Gonzalez, Matei Zaharia, Alexandros G. Dimakis, and Ion Stoica. 2026. Fantastic adaptive taxonomies and how to use them. *arXiv preprint arXiv:2607.16387*.

Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*.

Kathryn Chaloner and Isabella Verdinelli. 1995. Bayesian experimental design: A review. *Statistical Science*, 10(3):273–304.

Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. 2026. From failed trajectories to reliable LLM agents: Diagnosing and repairing harness flaws. *arXiv preprint arXiv:2606.06324*.

Peter I. Frazier and Warren B. Powell. 2010. Paradoxes in learning and the marginal value of information. *Decision Analysis*, 7(4):378–403.

Peter I. Frazier, Warren B. Powell, and Savas Dayanik. 2008. A knowledge-gradient policy for sequential information collection. *SIAM Journal on Control and Optimization*, 47(5):2410–2439.

Alain Hauser and Peter Bühlmann. 2014. Two optimal strategies for active learning of causal models from interventional data. *International Journal of Approximate Reasoning*, 55(4):926–939.

Shengran Hu, Cong Lu, and Jeff Clune. 2025. Automated design of agentic systems. In *International Conference on Learning Representations*.

Kirthevasan Kandasamy, Gautam Dasarathy, Jeff Schneider, and Barnabás Póczos. 2016. The multi-fidelity multi-armed bandit. In *Advances in Neural Information Processing Systems*.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia,

738	and Christopher Potts. 2024. DSPy: Compiling declarative language model calls into self-improving pipelines. In <i>International Conference on Learning Representations</i> .	792
739		793
740		794
741		795
742	Finnian Lattimore, Tor Lattimore, and Mark D. Reid. 2016. Causal bandits: Learning good interventions via causal inference. In <i>Advances in Neural Information Processing Systems</i> .	796
743		
744		
745		
746	Jiazheng Li, Emine Yilmaz, Bei Chen, and Dieu-Thu Le. 2026. Towards self-improving error diagnosis in multi-agent systems. <i>arXiv preprint arXiv:2604.17658</i> .	797
747		798
748		799
749		800
750	Xiaofeng Lin, Yingxu Wang, Tung Sum Thomas Kwok, Daniel Guo, Sahil Arun Nale, Charles Fleming, and Guang Cheng. 2026. REFLECT: Intervention-supported error attribution for silent failures in LLM agent traces. <i>arXiv preprint arXiv:2606.09071</i> .	801
751		
752		
753		
754		
755	Jiale Liu, Huajun Xi, Shaokun Zhang, Yifan Zeng, Tianwei Yue, Chi Wang, Jian Kang, Qingyun Wu, and Huazheng Wang. 2026. Who&when pro: Can LLMs really attribute failures in AI agents? <i>arXiv preprint arXiv:2607.09996</i> .	802
756		803
757		804
758		805
759		
760	Andrea Locatelli, Maurilio Gutzeit, and Alexandra Carpentier. 2016. An optimal algorithm for the thresholding bandit problem. In <i>Proceedings of the 33rd International Conference on Machine Learning</i> .	806
761		807
762		808
763		809
764	Ming Ma, Jue Zhang, Fangkai Yang, Yu Kang, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. 2025. DoVer: Intervention-driven auto debugging for LLM multi-agent systems. <i>arXiv preprint arXiv:2512.06749</i> .	810
765		811
766		812
767		
768		
769	Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-refine: Iterative refinement with self-feedback. In <i>Advances in Neural Information Processing Systems</i> .	813
770		814
771		815
772		816
773		817
774		818
775		819
776		820
777	Matthias Poloczek, Jialei Wang, and Peter I. Frazier. 2017. Multi-information source optimization. In <i>Advances in Neural Information Processing Systems</i> .	821
778		822
779		823
780	Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. 2023. Automatic prompt optimization with “gradient descent” and beam search. In <i>Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing</i> .	824
781		825
782		
783		
784		
785		
786	Daniel Russo. 2016. Simple Bayesian algorithms for best arm identification. In <i>Proceedings of the 29th Conference on Learning Theory</i> .	826
787		827
788		828
789	Jaineet Shah. 2026. Causal agent replay: Counterfactual attribution for LLM-agent failures. <i>arXiv preprint arXiv:2606.08275</i> .	829
790		
791		
	Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In <i>Advances in Neural Information Processing Systems</i> .	830
		831
		832
		833
	Edward K. Smith, Earl T. Barr, Claire Le Goues, and Yuriy Brun. 2015. Is the cure worse than the disease? overfitting in automated program repair. In <i>Proceedings of the 10th Joint Meeting on Foundations of Software Engineering</i> .	834
		835
		836
		837
	Simon Tong and Daphne Koller. 2001. Active learning for structure in Bayesian networks. In <i>Proceedings of the 17th International Joint Conference on Artificial Intelligence</i> .	838
		839
		840
	Rakshith Vasudev, Melisa Russak, Dan Bikel, and Waseem Alshikh. 2026. Accurate failure prediction in agents does not imply effective failure prevention. <i>arXiv preprint arXiv:2602.03338</i> .	841
		842
		843
		844
		845
	Abraham Wald. 1945. Sequential tests of statistical hypotheses. <i>The Annals of Mathematical Statistics</i> , 16(2):117–186.	
	Mengting Wan, Tara Safavi, Sujay Kumar Jauhar, Yujin Kim, Scott Counts, Jennifer Neville, Siddharth Suri, Chirag Shah, Ryen W. White, Longqi Yang, Reid Andersen, Georg Buscher, Dhruv Joshi, and Nagu Rangan. 2024. TnT-LLM: Text mining at scale with large language models. In <i>Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining</i> .	
	Xiaojun Wu, Cehao Yang, Honghao Liu, Xueyuan Lin, Wenjie Zhang, Zhichao Shi, Xuhui Jiang, Chengjin Xu, Jia Li, and Jian Guo. 2026. Bayesian-Agent: Posterior-guided skill evolution across LLM agent harnesses. <i>arXiv preprint arXiv:2606.08348</i> .	
	Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large language models as optimizers. In <i>International Conference on Learning Representations</i> .	
	Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. <i>arXiv preprint arXiv:2406.12045</i> .	
	Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. 2025. Optimizing generative AI by back-propagating language model feedback. <i>Nature</i> , 639.	
	Andreas Zeller and Ralf Hildebrandt. 2002. Simplifying and isolating failure-inducing input. <i>IEEE Transactions on Software Engineering</i> , 28(2):183–200.	
	Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. 2026. AgenTracer: Who is inducing failure in the LLM agentic systems? In <i>International Conference on Learning Representations</i> . ArXiv:2509.03312.	

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. 2025a. Darwin Gödel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*.

Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. 2025b. Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems. In *Proceedings of the 42nd International Conference on Machine Learning*. ArXiv:2505.00212.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Kunlun Zhu, Zijia Liu, Bingxuan Li, Muxin Tian, Yingxuan Yang, Jiaxun Zhang, Pengrui Han, Qipeng Xie, Fuyang Cui, Weijia Zhang, Xiaoteng Ma, Xiaodong Yu, Gowtham Ramesh, Jialian Wu, Zicheng Liu, Pan Lu, James Zou, and Jiaxuan You. 2025. Where LLM agents fail and how they can learn from failures. *arXiv preprint arXiv:2509.25370*.

Kunlun Zhu, Xuyan Ye, Zhiguang Han, Yuchen Zhao, Bingxuan Li, Weijia Zhang, Muxin Tian, Xiangru Tang, Pan Lu, James Zou, Jiaxuan You, and Heng Ji. 2026. AgentDebugX: An open-source toolkit for failure observability, attribution, and recovery in LLM agents. *arXiv preprint arXiv:2607.18754*.

A Proofs

Proposition 1. For any signal Y and any action set, $\max_S \mathbb{E}[U(S) \mid Y] \leq \mathbb{E}[\max_S U(S) \mid Y]$, so taking expectations gives $V(Y) \leq \mathbb{E}[\max_S U(S)] - \max_S \mathbb{E}[U(S)]$, the value of perfect information; this is the standard argument and needs no assumption on how Y relates to Δ . With $U(S) = \sum_{k \in S} \Delta_k$ and at most one patch per cell, both maxima separate over cells: $\max_S U(S) = \sum_{ij} \max_k \Delta_k^+$ and $\max_S \mathbb{E}[U(S)] = \sum_{ij} \max_k (\mathbb{E}\Delta_k)^+$, which gives V_{ij} ; no independence between cells is needed. For Eq. 3, let k^* maximise $\mathbb{E}\Delta_k$. If $\mathbb{E}\Delta_{k^*} \geq 0$, then $V_{ij} = \mathbb{E}[\max_k \Delta_k^+] - \mathbb{E}[\Delta_{k^*}^+] + \mathbb{E}[\Delta_{k^*}^-]$, which is Eq. 3 with equality. Otherwise $V_{ij} = \mathbb{E}[\max_k \Delta_k^+] = \mathbb{E}[\max_k \Delta_k^+ - \Delta_{k^*}^+] + \mathbb{E}[\Delta_{k^*}^+]$, and $\mathbb{E}[\Delta_{k^*}^+] < \mathbb{E}[\Delta_{k^*}^-]$ because $\mathbb{E}\Delta_{k^*} < 0$. Finally, the repair term $F f_{ie_{ij}q_k}$ is non-negative, so $\Delta_k^- \leq (1 - F)\rho_k^+ + c_a + c_w(1 - e_{ij})$; take expectations.

Corollary 1. With one patch, $V = \mathbb{E}[\Delta^+] - (\mathbb{E}\Delta)^+ = \pi G - (\pi G - (1 - \pi)H)^+ = \min(\pi G, (1 - \pi)H)$. The assumption that Δ is

a function of e_{ij} is what makes the edge state sufficient: if q_k or ρ_k is uncertain given e_{ij} , learning only e_{ij} leaves value on the table and the general expression applies.

Proposition 2. The first part is Wald’s approximation for a sequential probability ratio test between $p_1 = b + (1 - b)q$ and $p_0 = b$: with thresholds $\pm \log \frac{1-\delta}{\delta}$, neglecting overshoot, the expected sample size under p_1 is the threshold divided by the per-observation drift $\text{KL}(p_1 \parallel p_0)$. The second is the normal-approximation sample size of a one-sided test of a binomial proportion, evaluated with the variance at r_0 .

B Reproducibility

An anonymised repository accompanies the submission. It contains the environments and fault suites (agent_exp/env.py, agent_exp/tau2_env.py), every prompt used for attribution, taxonomy induction, labelling, patch generation and judging (agent_exp/analyst.py), the agent and analyst settings (DeepSeek-V4-Flash and DeepSeek-V4-Pro through the provider’s Anthropic-compatible endpoint, at most 4,000 output tokens per agent call and 8,000 per structured analyst call, provider-default sampling), and the complete intervention banks (runs/tau2_*/bank.json: every replay, judge check, null replay and regression run with its token count and source trace). Bank construction is deterministic given the seeds in agent_exp/bank.py: cells are the attributed cells in order of blame plus random unattributed cells of populated categories, and replay traces and regression tasks are drawn with fixed seeds. Every table in the paper is produced offline from the banks by agent_exp.reanalysis, agent_exp.heldout_breakdown, agent_exp.review2_report and agent_exp.report_tau2; the new runs of Appendix E, the unpatched baseline and the airline held-out runs come from agent_exp.review2_runs and agent_exp.bank_heldout; the README lists the commands. Re-collecting the banks needs API access and is not bit-reproducible, because the provider does not offer seeded sampling.

C Significance-Thresholded Scoring

Charging only regressions significant against the paired-trial r_0 reproduces the picture of a scoring

Budget (episodes)	τ^2 retail (development)						τ^2 airline (held out)					
	significant harm			+ $c_w=0.02$			significant harm			+ $c_w=0.02$		
	10	40	80	10	40	80	10	40	80	10	40	80
Apply attributed	0.84	0.84	0.84	0.16	0.16	0.16	0.60	0.60	0.60	0.15	0.15	0.15
Replay-each	0.78	0.74	0.71	0.14	0.33	0.36	0.43	0.09	0.10	0.07	-0.12	-0.08
HarnessFix-style, per patch	0.52	0.51	0.52	0.52	0.42	0.30	0.18	0.40	0.43	0.18	0.37	0.31
HarnessFix-style, bundle	0.11	0.21	0.26	-0.21	-0.31	-0.31	0.34	0.53	0.52	0.07	0.12	0.11
Uncertainty sampling	0.42	0.50	0.31	0.01	0.23	0.01	0.56	0.50	0.51	0.28	0.25	0.25
Uncertainty + checks	0.14	0.47	0.48	-0.25	0.25	0.24	0.55	0.50	0.53	0.27	0.29	0.36
CARVE, replay only	0.01	0.18	0.14	-0.48	-0.26	-0.39	0.22	0.35	0.48	-0.16	-0.02	0.12
CARVE	0.33	0.32	0.52	-0.01	0.19	0.47	0.55	0.34	0.25	0.30	0.23	0.19
CARVE, measured check	0.01	0.18	0.14	-0.48	-0.26	-0.39	0.25	0.43	0.54	-0.09	0.07	0.20
CARVE, robust check	0.34	-0.03	0.02	-0.15	-0.49	-0.48	0.22	0.34	0.48	-0.12	-0.01	0.14
Net-effect (ours)	0.24	0.52	0.66	0.09	0.41	0.52	0.26	0.39	0.46	0.14	0.31	0.36

Table 6: Sensitivity analysis: as Table 1, but accepted patches are charged only their significant measured regressions (one-sided binomial $p < 0.05$ against r_0), mean over 200 resampling seeds of the original bank. This scoring sets uncertain harm to zero and so favours policies that apply many patches.

that sets uncertain harm to zero: applying everything recovers 84% and 60% of the oracle and no verifying policy reaches it. With $c_w = 0.02$ per wrong change, the per-patch HarnessFix-style rule (retail, budgets 10–40; airline, 40) and the net-effect method (retail, 80; airline, 80, tied with uncertainty sampling with checks) lead, and CARVE declines with budget on airline (0.30→0.19). Appendix D varies c for both cost types; the winner changes with c .

D Change-Cost Sweep

Table 7 varies the change cost. The best policy changes with c : applying the attributed patches is best on retail at $c = 0$ (on airline, CARVE told $c = 0$ edges it, 0.71 against 0.60), replay-each at $c = 0.01$ on retail, the per-patch HarnessFix-style rule from $c = 0.01$ to 0.02 on airline and at 0.02 on retail, and the net-effect method at $c = 0.05$ on retail under both cost types and on airline per applied change. Charging every applied change rather than only wrong ones lowers every policy but changes the winner in only one setting (airline, $c = 0.05$). Told $c = 0$, the policies that use a cost internally accept patches with significant regressions, which is why some of them are negative in the first column.

E Held-Out Ablations

These runs were made on retail, on the same held-out episodes as Table 5 (Tables 8 and 9) or on new clean episodes (Table 10). Differences are paired and task-clustered, with 90% intervals. Leaving any one of the seven attributed patches out lowers

success by 0.00 to 0.03, and the sum of these contributions is not distinguishable from the whole set’s effect, so the runs neither show nor exclude interactions. On clean episodes, applying everything gains +0.04 [−0.03, 0.10] over the unpatched agent. The bundle gate is ahead of it by +0.04 [0.01, 0.08], but three of its five seed sets are identical to the attributed set and one is empty, so the difference is not an effect of verification; on the 72 slots where it ran the identical set the difference is +0.03 [−0.04, 0.09], which indicates the run-to-run noise of these comparisons. Unpatched clean success was 0.82 both before and after the new runs.

F Development Variants

On the retail bank we tried, before any airline evaluation of them:

- CARVE with two null replays per category before trusting its replays (per-category b);
- CARVE with a regression gate of two runs per accepted patch;
- both combined;
- an evidence rule that requires a positive likelihood ratio from data;
- tempered and flat priors;
- a Bayesian regression gate.

With control replays and the gate combined, CARVE gained 0.68 and 0.60 of the oracle at budgets 10 and 20, above plain CARVE (0.42, 0.48), but fell below it at 40 and 80 (0.47 vs. 0.57 and 0.56). It never beat the per-patch HarnessFix-style rule (0.72, 0.73). Post hoc on airline it was worse

Policy (budget 40)		per wrong change c_w				per applied change c_a			
		0	0.01	0.02	0.05	0	0.01	0.02	0.05
Retail	Apply attributed	0.84	0.50	0.16	-0.85	0.84	0.43	0.01	-1.23
	Replay-each	0.74	0.53	0.33	-0.30	0.74	0.48	0.22	-0.57
	HarnessFix-style, per patch	0.51	0.46	0.42	0.28	0.51	0.42	0.34	0.09
	HarnessFix-style, bundle	0.21	-0.05	-0.31	-1.09	0.21	-0.11	-0.42	-1.38
	Uncertainty sampling	-1.90	-2.46	0.23	0.05	-1.90	-2.57	0.09	-0.17
	Uncertainty + checks	-1.57	-1.95	0.25	0.25	-1.57	-2.06	0.12	0.01
	CARVE, replay only	-0.07	-0.49	-0.26	0.00	-0.07	-0.60	-0.44	-0.25
	CARVE	-0.19	-0.19	0.19	0.28	-0.19	-0.27	0.07	0.14
	CARVE, measured check	-0.07	-0.49	-0.26	0.00	-0.07	-0.60	-0.44	-0.25
	CARVE, robust check	-0.05	-0.70	-0.49	0.01	-0.05	-0.81	-0.66	-0.26
	Net-effect (ours)	-1.02	-0.58	0.41	0.63	-1.02	-0.67	0.32	0.44
Airline	Apply attributed	0.60	0.37	0.15	-0.53	0.60	0.26	-0.08	-1.09
	Replay-each	0.09	-0.02	-0.12	-0.43	0.09	-0.08	-0.25	-0.76
	HarnessFix-style, per patch	0.40	0.39	0.37	0.32	0.40	0.33	0.25	0.02
	HarnessFix-style, bundle	0.53	0.32	0.12	-0.50	0.53	0.22	-0.09	-1.02
	Uncertainty sampling	0.49	0.09	0.25	0.19	0.49	-0.04	0.01	-0.25
	Uncertainty + checks	0.44	0.01	0.29	0.21	0.44	-0.12	0.08	-0.13
	CARVE, replay only	0.48	0.08	-0.02	0.06	0.48	-0.07	-0.26	-0.44
	CARVE	0.71	0.33	0.23	0.10	0.71	0.22	0.08	-0.07
	CARVE, measured check	0.56	0.10	0.07	0.11	0.56	-0.04	-0.17	-0.38
	CARVE, robust check	0.63	0.21	-0.01	0.08	0.63	0.06	-0.26	-0.42
	Net-effect (ours)	0.60	0.10	0.31	0.29	0.60	-0.04	0.14	0.05

Table 7: Value at budget 40 as a fraction of the repair-only oracle, with significant regressions charged, as the change cost varies. Policies that use a cost internally are told the same c (the net-effect method always charges it per applied change, the others per wrong change). At $c = 0$ the uncertainty-based policies accept almost every cell with a positive posterior and pick up patches with significant regressions.

Set (seed 0)	With – without step patch			Without – unpatched
	All	No step fault	Step fault	No step fault
Apply attributed	0.03 [-0.04, 0.10]	-0.03 [-0.10, 0.04]	0.58 [0.37, 0.80]	0.09 [0.01, 0.18]
HarnessFix-style, bundle	0.03 [-0.04, 0.10]	-0.03 [-0.10, 0.04]	0.58 [0.37, 0.80]	0.09 [0.01, 0.18]
Uncertainty sampling	0.01 [-0.07, 0.08]	-0.02 [-0.09, 0.06]	0.25 [-0.07, 0.57]	0.06 [-0.04, 0.15]
CARVE	0.03 [-0.04, 0.11]	0.00 [-0.07, 0.07]	0.33 [0.07, 0.59]	0.05 [-0.03, 0.13]
Net-effect (ours)	0.05 [-0.02, 0.12]	-0.01 [-0.08, 0.06]	0.58 [0.34, 0.83]	0.02 [-0.07, 0.10]

Table 8: Removing the step-limit patch from each policy’s seed-0 set (the bundle gate’s seed-0 set equals the attributed set). 108 episodes without and 12 with the step-limit fault.

Patch left out of Apply attributed	All	No step fault
CFG.max_steps	0.03 [-0.04, 0.10]	-0.03 [-0.10, 0.04]
POL.cancel	0.03 [-0.04, 0.10]	0.04 [-0.04, 0.12]
POL.exchange	0.03 [-0.04, 0.09]	0.03 [-0.05, 0.11]
POL.generic	0.03 [-0.04, 0.09]	0.01 [-0.05, 0.07]
POL.intro	0.02 [-0.05, 0.08]	0.02 [-0.05, 0.08]
POL.modify	0.03 [-0.05, 0.10]	0.04 [-0.05, 0.12]
POL.return	0.00 [-0.06, 0.06]	-0.01 [-0.08, 0.06]
Whole set vs unpatched	0.12 [0.05, 0.19]	0.06 [-0.01, 0.14]
Whole set – sum of parts	-0.04 [-0.35, 0.27]	-0.03 [-0.38, 0.33]

Table 9: Leave-one-out on the attributed set: success of the whole set minus the set without the named patch.

than plain CARVE (0.19 vs. 0.31 at budget 10). The other variants were worse everywhere; the evidence rule and the Bayesian gate accepted almost nothing (gain ≤ 0.12). These numbers use the fixed change cost c without regressions and the bank before the regression runs were extended. The net-effect method of §5 was developed afterwards,

	Old	New	Pooled	– Unpatched	– Apply
Unpatched	0.82	0.82	0.82	–	-0.04 [-0.10, 0.03]
Apply attributed	0.87	0.85	0.85	0.04 [-0.03, 0.10]	
Net-effect (ours)	0.83	0.87	0.86	0.04 [-0.01, 0.09]	0.00 [-0.05, 0.06]
HarnessFix-style, bundle	0.87	0.91	0.90	0.08 [0.03, 0.14]	0.04 [0.01, 0.08]
HarnessFix-style, per patch	0.90	0.85	0.86	0.05 [-0.01, 0.10]	0.01 [-0.06, 0.08]

Table 10: Clean held-out episodes: 38 old (Table 5) and 120 new (40 tasks $\times$ 3), where new episode e uses the policy’s seed- $(e \bmod 5)$ set. Differences pool both, paired by episode or slot and clustered by task.

on retail, from the diagnosis that CARVE lacked per-category spurious recovery and evidence for regressions.

G ShopDesk Details

ShopDesk has six deterministic tools (orders, products, policies, refunds), answers checked exactly (refund amounts, counts, totals), and five injected faults: a tool description misstating price units, a list tool whose description omits pagination, a format instruction that rounds to whole dollars, a planning instruction that skips the policy lookup, and a tight step limit. Over three seeds, the analyst’s attribution accuracy on injected failures was 0.75, 0.64 and 0.77. ShopDesk was evaluated online, without a bank: each policy ran against the live agent with fault-based ground truth (an edge is

real when at least 30% of a category’s failures come from that component’s fault). Table 11 reports edge F1 and held-out success. Large harms do occur: a formatting patch that passed its target-category replays made 26% of held-out tasks answer N/A (success 0.89→0.73).

H The CARVE Design Study

Why the simulator was optimistic. Our first design study, CARVE, used a simulator in which it needed half the budget of the strongest baseline (Appendix H). Calibrated to the measured judge accuracy, spurious recovery, analyst accuracy and cost ratio, the simulator still predicts that CARVE leads uncertainty sampling by 14 points on retail at budget 50; on the bank it trails by 15 points at budget 40 (significant harm, $c_w = 0.02$). The simulator shared the method’s structure: one b for all categories, value spread over many edges, and a fixed cost c that made every wrong change harmful, an assumption that was never measured and on which the value of verification largely depends.

Our first approach treated judge checks as cheap and replays as expensive, and allocated a budget over both by Bayesian experimental design. We keep it here as the multi-fidelity method compared in §7, together with the simulator in which it was developed. The simulator shares the method’s structure. Its results show the method working under its own assumptions, and they are not evidence about real agents.

Calibrated simulation. With the measured judge accuracy, spurious recovery, analyst accuracy and cost ratio, the simulator predicts that CARVE runs no judge checks on ShopDesk and retail and only a few on airline, which matches the break-even analysis in Table 2. For τ^2 retail it predicts net gains at budget 50 of 0.51 for CARVE and 0.38 for uncertainty sampling; for airline, 0.33 and 0.31; for ShopDesk, 0.81 and 0.79.

H.1 Problem Formulation

An agent is built from *components* $j \in \{1, \dots, J\}$: sections of its system prompt, tool descriptions, tool implementations, orchestration settings. Its failures have been grouped into *categories* $i \in \{1, \dots, I\}$ by an LLM, with f_i the share of failures in category i . For every cell (i, j) an analyst LLM can propose K candidate patches a_{ijk} that rewrite component j to address category i .

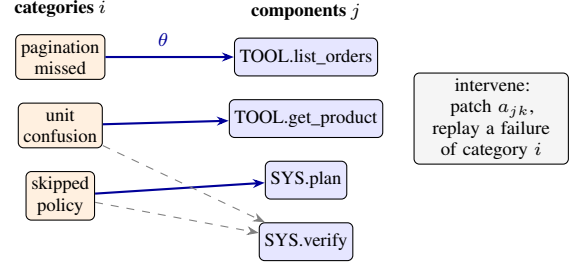


Figure 1: The category–component graph. Solid edges are true causal edges; dashed ones are plausible but wrong attributions (e.g. blaming verification for a unit error). Each intervention patches one component and replays one failure.

Interventions. An intervention (i, j, k, ϕ) samples a failed trajectory labelled i , applies patch a_{ijk} , and observes a binary outcome at fidelity ϕ . A *full replay* ($\phi = F$) resumes the episode from the attributed failing step and records whether the task now succeeds. A *single-step check* ($\phi = S$) regenerates only the failing step and asks a judge whether the diagnosed error is gone. They cost c_F and $c_S \ll c_F$.

Edges and patch quality. We write $e_{ij} \in \{0, 1\}$ for whether component j is a cause of category i , meaning some change to j repairs a non-trivial fraction of its failures, and $q_{ijk} \in [0, 1]$ for the repair probability of patch k when $e_{ij} = 1$. This separation matters because a negative replay outcome is ambiguous: either j is the wrong component or a_{ijk} is a bad patch. Testing several patches per cell resolves the ambiguity.

Objective. After spending at most budget B , the method accepts a set of cells $\mathcal{A}$ and, for each, a patch k_{ij}^* . Accepting a true edge repairs a fraction q_{ijk^*} of category i ’s failures; accepting a false edge costs a regression c_{fp} , reflecting that unnecessary changes perturb behaviour on tasks that used to succeed (Vasudev et al., 2026). We measure

$$G(\mathcal{A}) = \sum_i f_i \left(1 - \prod_{j \in \mathcal{A}_i^+} (1 - q_{ijk_{ij}^*}) \right) - c_{fp} |\{(i, j) \in \mathcal{A} : e_{ij} = 0\}|, \quad (4)$$

where $\mathcal{A}_i^+ = \{j : (i, j) \in \mathcal{A}, e_{ij} = 1\}$. This is the net failure mass repaired; we report it as a fraction of the oracle value obtained by accepting exactly the true edges with their best patches.

Method	Edge F1 at budget				Held-out success @60	Single checks
	10	20	30	60		
LLM attribution only	0.61 $\pm$ 0.04	0.61 $\pm$ 0.04	0.61 $\pm$ 0.04	0.61 $\pm$ 0.04	0.98 $\pm$ 0.00	0
Replay-each (do-then-verify)	0.62 $\pm$ 0.03	0.63 $\pm$ 0.03	0.67 $\pm$ 0.05	0.75 $\pm$ 0.03	0.99 $\pm$ 0.00	0
Uncertainty sampling	0.55 $\pm$ 0.04	0.57 $\pm$ 0.04	0.58 $\pm$ 0.05	0.66 $\pm$ 0.04	0.99 $\pm$ 0.00	0
Uncertainty + cheap checks	0.52 $\pm$ 0.07	0.56 $\pm$ 0.07	0.53 $\pm$ 0.09	0.53 $\pm$ 0.08	0.96 $\pm$ 0.03	57
CARVE, full replay only	0.61 $\pm$ 0.07	0.61 $\pm$ 0.06	0.59 $\pm$ 0.04	0.64 $\pm$ 0.03	0.91 $\pm$ 0.09	0
CARVE (default check model)	0.59 $\pm$ 0.06	0.57 $\pm$ 0.05	0.61 $\pm$ 0.05	0.68 $\pm$ 0.03	0.98 $\pm$ 0.01	4
CARVE, calibrated check	0.59 $\pm$ 0.06	0.58 $\pm$ 0.05	0.60 $\pm$ 0.05	0.66 $\pm$ 0.04	0.89 $\pm$ 0.08	0
CARVE, robust check model	0.58 $\pm$ 0.07	0.61 $\pm$ 0.06	0.61 $\pm$ 0.04	0.69 $\pm$ 0.01	0.99 $\pm$ 0.01	0
Unpatched agent	0.59 $\pm$ 0.04					

Table 11: ShopDesk (3 seeds, mean $\pm$ s.e.). Held-out success after applying the patches accepted at budget 60 to 120 held-out episodes.

H.2 CARVE

Observation model For a trajectory labelled i and patch a_{ijk} , the probability that a full replay succeeds is

$$p_{ijk} = \begin{cases} \lambda q_{ijk} + (1 - \lambda) b & e_{ij} = 1, \\ b & e_{ij} = 0, \end{cases} \quad (5)$$

where b is the spurious recovery rate (stochastic re-generation sometimes succeeds regardless of the patch) and λ is the accuracy of the category labels used to sample trajectories; a mislabelled trajectory is treated as unaffected by the patch. A single-step check returns $z = 1$ with probability $sp + \epsilon(1 - p)$, where s and ϵ are the sensitivity and false-positive rate of the check relative to the full outcome. All four nuisance parameters can be estimated from a small calibration set on which both fidelities and a human audit are run.

Attribution-informed prior Let n_{ij} be the number of failures in category i that the analyst attributed to component j , and $\hat{r}$ the analyst’s attribution accuracy estimated on a small audited sample. With d the expected number of causal components per category and $\rho = d/J$ the base edge density, we set

$$\pi_{ij} = (1 - \hat{r})\rho + \hat{r} \min(1, d n_{ij}/n_i), \quad (6)$$

so that an unreliable analyst ($\hat{r} \rightarrow 0$) yields the flat prior ρ and a reliable one concentrates mass on the components it names. Eq. 6 is a heuristic interpolation, not a calibrated model; with a larger audit, π could instead be fitted by logistic regression on audited attributions. Patch quality has prior $q_{ijk} \sim \text{Beta}(\alpha_q, \beta_q)$.

Posterior Cells are independent given the data. For each cell we keep the posterior log-odds of e_{ij}

and, for each patch, a posterior over q_{ijk} on a fixed grid of $G = 101$ points. With $\mathcal{D}_{ijk}$ the outcomes observed for patch k ,

$$\text{logit } P(e_{ij}=1 \mid \mathcal{D}) = \text{logit } \pi_{ij} + \sum_k \log(\bar{L}_{ijk}^1/L_{ijk}^0), \quad (7)$$

where $\bar{L}_{ijk}^1 = \int P(\mathcal{D}_{ijk} \mid q, e_{ij}=1) dP(q)$ is the marginal likelihood of patch k ’s outcomes under a true edge and $L_{ijk}^0 = P(\mathcal{D}_{ijk} \mid e_{ij}=0)$. Every update multiplies one grid vector by a likelihood vector, so updates and look-aheads are vectorised dot products.

Selecting interventions The value of the current decision for cell (i, j) , with P_{ij} its edge posterior and $\hat{Q}_{ij} = \max_k \mathbb{E}[q_{ijk} \mid \mathcal{D}]$, is

$$V_{ij} = \max(P_{ij} f_i \hat{Q}_{ij} - (1 - P_{ij}) c_{\text{fp}}, 0), \quad (8)$$

the expected contribution of the cell to Eq. 4 under the Bayes decision (accept iff the first term is positive), treating cells as additive. Eq. 4 combines components of one category multiplicatively, so V_{ij} slightly overstates the value of a second cause for an already repaired category; we accept this approximation because it keeps cells independent. For an action $a = (i, j, k, \phi)$ repeated n times, let $V'_{ij}(s)$ be the value after observing s successes. The n -step knowledge gradient is

$$\text{KG}_n(a) = \sum_{s=0}^n P(s \mid \mathcal{D}, a) V'_{ij}(s) - V_{ij}, \quad (9)$$

and CARVE runs the action maximising

$$\text{score}(a) = \max_{n \in \{1, 2, 4, 8\}} \frac{\text{KG}_n(a)}{n c_\phi}. \quad (10)$$

Looking ahead several observations matters because a single binary outcome often cannot flip

a decision, in which case $KG_1 = 0$ even for a valuable cell; maximising over n is the standard remedy (Frazier et al., 2008). Because cells are independent, only the observed cell’s scores change after each outcome, so each step rescores one cell in $O(KG)$ time after an initial pass over all cells.

Decisions and stopping. At any point the method accepts the cells whose current value is positive and applies, for each, the patch with the highest posterior mean quality. It stops early when no action has a score above a small threshold, i.e. when no remaining experiment is expected to be worth its cost.

H.3 When Do Priors and Cheap Checks Help?

These results analyse an idealised per-cell sequential test, not CARVE’s full policy; they explain which quantities govern the method’s behaviour. We consider a single cell and a single patch with known success probabilities p_1 (true edge) and $p_0 = b$, under a rule that stops when the posterior log-odds leaves $[-A, A]$, $A = \log \frac{1-\delta}{\delta}$. Let $D_1 = \text{KL}(p_1 \| p_0)$ and $D_0 = \text{KL}(p_0 \| p_1)$ be the expected log-likelihood-ratio drift per observation under each hypothesis. Proofs, which follow Wald’s approximations (Wald, 1945), are in Appendix H.5.

Proposition 3 (Value of the LLM prior). *Let $|\ell| < A$ and neglect overshoot and stopping at the wrong boundary (whose probability is approximately $\delta e^{-\ell}$ for a true edge and δe^{ℓ} for a false one). A cell with initial log-odds ℓ then needs $N_1(\ell) = (A - \ell)/D_1$ observations in expectation if it is a true edge and $N_0(\ell) = (A + \ell)/D_0$ if not. Replacing a flat prior ℓ_ρ by LLM-derived log-odds ℓ_{ij} saves, in expectation over cells,*

$$\Delta = \frac{\rho \mathbb{E}[\delta \ell \mid e=1]}{D_1} - \frac{(1 - \rho) \mathbb{E}[\delta \ell \mid e=0]}{D_0}, \quad (11)$$

with $\delta \ell = \ell_{ij} - \ell_\rho$, observations per cell, which is positive exactly when the prior raises true edges more, relative to their base rate and drift, than it raises false ones.

Under Eq. 6, $\ell_{ij} \rightarrow \ell_\rho$ as $\hat{r} \rightarrow 0$, so $\Delta \rightarrow 0$: a useless analyst costs nothing beyond the flat prior, whereas trusting its labels outright (LLM-only) makes permanent errors. A miscalibrated analyst can make Δ negative, but only by an amount linear in the log-odds shift, after which the evidence takes over. Appendix H.7 compares these predictions with simulated sequential tests.

Proposition 4 (Cheap checks). *Let D_e^S denote the drifts of single-step outcomes, $z \sim \text{Bern}(s p_e + \epsilon(1 - p_e))$. Under the same approximations, a test using only single-step checks reaches the stopping boundary at lower expected cost than one using only full replays if and only if*

$$\frac{c_S}{c_F} < \frac{D_e^S}{D_e}, \quad (12)$$

where e is the true state of the cell.

Since e is unknown, the proposition gives a clear rule only when the cost ratio lies outside the interval spanned by the two critical ratios; inside it, which fidelity is better depends on the posterior, which is what CARVE’s look-ahead weighs.

With our default parameters ($p_1 = 0.54$, $b = 0.05$, $s = 0.85$, $\epsilon = 0.15$) the critical ratio is 0.31 for true edges and 0.44 for false ones; with a weak check ($s = 0.65$, $\epsilon = 0.35$) it falls to 0.05–0.08. Section H.4 tests both predictions.

H.4 Controlled Experiments

Real agents do not reveal which component truly caused a failure, so we first evaluate in a testbed where the causal graph is known and the method’s model is deliberately misspecified.

Setup

Testbed. Each world has $I = 12$ categories with Zipfian frequencies, $J = 20$ components and $K = 3$ patches per cell. Every category has one or two true components and one *decoy* component that wrong attributions prefer, mimicking symptom-level mislabelling. Patch quality is bimodal: with probability 0.3 a patch is bad ($q \sim \text{Beta}(1, 9)$), otherwise $q \sim \text{Beta}(6, 3)$. A trajectory sampled for category i is mislabelled with probability $1 - \lambda = 0.2$, in which case it comes from another category (weighted by frequency) and is repaired only if its own edge to j exists, unlike the model’s flat treatment of mislabelled traces. The analyst attributes 20 failures per category with accuracy $r = 0.5$; wrong attributions go to the decoy with probability 0.6. The analyst’s accuracy is estimated from an audit of 20 attributions. Costs are $c_F = 1$, $c_S = 0.1$, $c_{fp} = 0.02$, and $s = 0.85$, $\epsilon = 0.15$, $b = 0.05$. Section H.4 relaxes the assumptions the generator shares with the model. Full details are in Appendix H.6.

Method	Budget (full-replay units)			
	20	50	100	200
LLM attribution only	0.297 $\pm$ 0.021	0.297 $\pm$ 0.021	0.297 $\pm$ 0.021	0.297 $\pm$ 0.021
Replay-each (do-then-verify)	0.255 $\pm$ 0.021	0.270 $\pm$ 0.021	0.466 $\pm$ 0.027	0.468 $\pm$ 0.026
Replay-each, 3 patches	0.301 $\pm$ 0.025	0.317 $\pm$ 0.026	0.401 $\pm$ 0.026	0.612 $\pm$ 0.026
Sequential-each (adaptive stopping)	0.426 $\pm$ 0.016	0.522 $\pm$ 0.018	0.643 $\pm$ 0.016	0.813 $\pm$ 0.014
Thompson sampling	0.407 $\pm$ 0.020	0.419 $\pm$ 0.020	0.500 $\pm$ 0.017	0.625 $\pm$ 0.014
Uncertainty sampling	0.455 $\pm$ 0.016	0.581 $\pm$ 0.019	0.690 $\pm$ 0.017	0.804 $\pm$ 0.016
Uncertainty + cheap checks	0.549 $\pm$ 0.018	0.662 $\pm$ 0.016	0.784 $\pm$ 0.015	0.876 $\pm$ 0.009
CARVE (all components)	0.666 $\pm$ 0.019	0.787 $\pm$ 0.015	0.853 $\pm$ 0.011	0.902 $\pm$ 0.009
full replay only	0.514 $\pm$ 0.016	0.609 $\pm$ 0.020	0.697 $\pm$ 0.017	0.825 $\pm$ 0.012
myopic ($m=1$ look-ahead)	0.591 $\pm$ 0.016	0.652 $\pm$ 0.014	0.706 $\pm$ 0.016	0.729 $\pm$ 0.014
uninformed prior	0.347 $\pm$ 0.019	0.474 $\pm$ 0.017	0.631 $\pm$ 0.015	0.800 $\pm$ 0.012
one patch per cell	0.571 $\pm$ 0.019	0.636 $\pm$ 0.020	0.673 $\pm$ 0.019	0.688 $\pm$ 0.020
no label-noise term	0.663 $\pm$ 0.020	0.772 $\pm$ 0.018	0.864 $\pm$ 0.012	0.922 $\pm$ 0.006

Table 12: Net gain as a fraction of the oracle (mean $\pm$ s.e., 50 worlds). The lower block ablates CARVE.

Baselines. *LLM-only* accepts, for every category, the most-attributed component and any component holding at least 40% of the attributions, with the first patch, i.e. it acts on an induced taxonomy without verification. *Replay-each* tests attributed hypotheses in order of attribution count with three full replays of the first patch and accepts a hypothesis if at least two succeed, mirroring do-then-verify methods such as DoVer; *Replay-each, 3 patches* gives each hypothesis two replays of each of three patches and accepts if any patch succeeds twice; *Sequential-each* replays each hypothesis, cycling through patches, until CARVE’s own posterior leaves $[0.05, 0.95]$. Untested hypotheses fall back to LLM-only, except for *Sequential-each*, which uses the Bayes decision. *Thompson sampling* samples (e, q) from the posterior and replays the cell and patch with the largest sampled repair. *Uncertainty sampling* replays the cell with the largest frequency-weighted uncertainty $f_i \min(P_{ij}, 1 - P_{ij})$, cycling through patches, at full fidelity; *Uncertainty + cheap checks* first runs three single-step checks per (cell, patch) and then confirms with full replays. All baselines except LLM-only and the *Replay-each* variants share CARVE’s posterior and decision rule, so comparisons among them isolate the selection rule.

Metrics. Net gain (Eq. 4) as a fraction of the oracle, and edge F1. We report means and standard errors over 50 worlds for the main experiment and 30 for the others; all methods see the same worlds, so differences are paired.

Main results Figure 2 and Table 12 show that CARVE is best at every budget. With 50 units it recovers 79% of the oracle gain; the strongest baseline, uncertainty sampling with the same cheap checks, recovers 66% and needs about twice the budget (100 units, 78%) to match it. The gap nar-

rows at large budgets (90% vs. 88% at 200), as every reasonable method eventually resolves the frequent categories.

Where the gain comes from. Two comparisons separate the selection rule from the cheap checks. With full replays only, CARVE beats uncertainty sampling by 1–6 points (e.g. 0.609 vs. 0.581 at budget 50): the knowledge-gradient rule alone helps only modestly. Given cheap checks, its advantage grows to 7–13 points (0.787 vs. 0.662 at 50), because it decides per action whether a check is informative enough for its price and stops checking a cell once checks can no longer change the decision, whereas a fixed check-then-confirm schedule wastes checks on resolved cells. Most of CARVE’s advantage is therefore in *how it uses* cheap checks, which also makes their calibration critical (§H.4).

Verification baselines. LLM-only is stuck at 30% because it accepts decoys and bad first patches. *Replay-each* is initially *worse* than not verifying: with $p_1 \approx 0.54$, requiring two successes in three replays rejects about 44% of true edges even when the patch is good, and its fixed per-hypothesis cost exhausts small budgets on frequent but already-certain hypotheses. Testing three patches (61% at 200) or stopping adaptively (81% at 200) helps, but both remain well below CARVE at small budgets (32% and 52% at 50). Thompson sampling spends most replays on cells it already believes in and reaches only 63% at 200.

Ablations. Removing single-step checks costs 15–18 points at small budgets. Restricting the look-ahead to one observation is below the full method at every budget and plateaus at 73%, because many cells cannot be flipped by a single outcome and are never revisited. A flat prior costs 32 points at budget 20 and still 10 points at 200. With one patch per cell the method plateaus at 69%: it cannot tell a wrong component from a bad patch and rejects true edges for good. Dropping the label-noise term makes no difference at the default label accuracy $\lambda = 0.8$ (it is even slightly better at large budgets, 0.922 vs. 0.902 at 200), but it matters when labels are worse (§H.4).

When does the LLM prior help? Figure 3 varies the analyst’s accuracy r . As Proposition 3 suggests, CARVE improves monotonically with r and is never below the flat-prior variant: at $r = 0.1$ it scores 0.51 ± 0.02 against 0.48 ± 0.03 . The prior is not exactly flat there (the decoy still receives

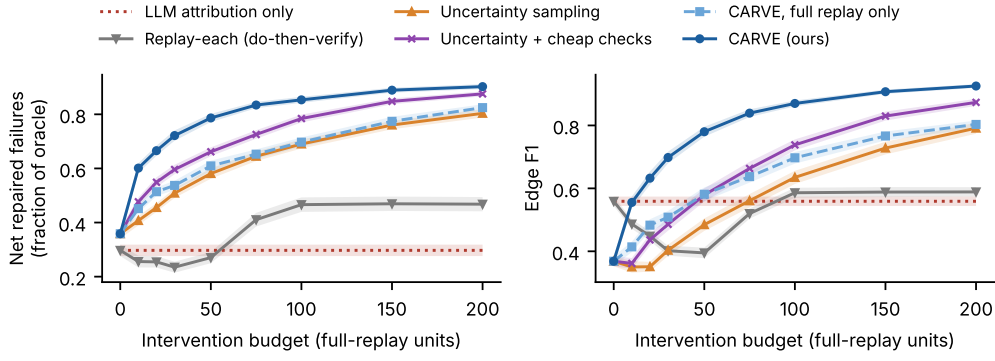


Figure 2: Net repaired failure mass (left) and edge F1 (right) against intervention budget, 50 worlds, shaded ± 1 s.e.

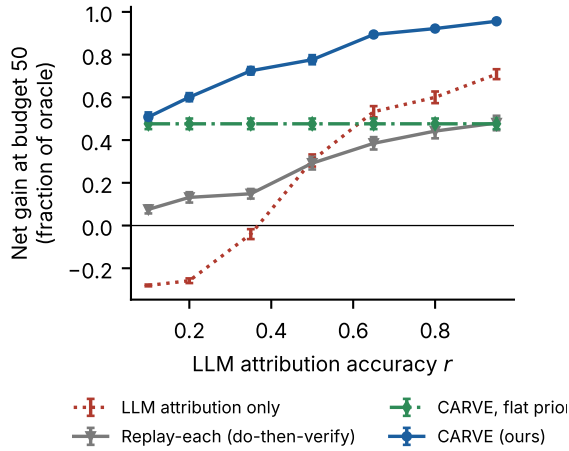


Figure 3: Net gain at budget 50 as the analyst’s attribution accuracy r varies (30 worlds, ± 1 s.e.).

about twice the base rate), but its shifts are small, so evidence quickly dominates. Acting on attributions directly is harmful below $r \approx 0.35$, where LLM-only has a *negative* net gain (regressions outweigh repairs), and even at $r = 0.95$ it reaches only 71% of the oracle against 96% for CARVE, since it cannot detect bad patches. Replay-each improves with r and only matches the flat-prior variant of CARVE at $r = 0.95$ (0.48 vs. 0.48).

When do cheap checks help? Figure 5 (Appendix H.8) is qualitatively consistent with Proposition 4. With the default check, the benefit of cheap checks shrinks as their cost grows and vanishes between 0.35 and 0.5 (0.64 ± 0.02 vs. 0.61 ± 0.03 at 0.35, overlapping; 0.59 vs. 0.61 at 0.5), in line with the predicted critical ratios of 0.31 and 0.44. At cost 0.1, a check of quality $s - \epsilon = 0.5$ (critical ratio 0.14–0.22) still helps, while one of quality 0.3 (critical ratio 0.05–0.08) brings no gain. At cost

Condition (budget 50)	Unc. + checks	CARVE
Default testbed	0.656 ± 0.022	0.776 ± 0.021
Label accuracy $\lambda = 0.5$	0.513 ± 0.023	0.688 ± 0.023
Analyst overconfident ($\hat{r} + 0.3$)	0.752 ± 0.018	0.857 ± 0.014
Analyst underconfident ($\hat{r} - 0.3$)	0.495 ± 0.023	0.653 ± 0.032
Judge fooled by bad patches	0.579 ± 0.022	0.377 ± 0.027
Regression cost $c_{fp} = 0.005$	0.711 ± 0.022	0.798 ± 0.016
Regression cost $c_{fp} = 0.05$	0.643 ± 0.024	0.793 ± 0.019
Regression cost $c_{fp} = 0.1$	0.624 ± 0.023	0.823 ± 0.018

Table 13: Violations of the model’s assumptions (30 worlds). The judge-fooled row is the one setting where CARVE loses.

0.5 CARVE is slightly below its full-replay-only variant although its action set contains every full-replay action, which shows that the greedy rule is not exactly cost-optimal when the two fidelities are nearly equally efficient.

Robustness The generator shares several assumptions with the model: check errors depend only on the full outcome, cells are independent, the analyst’s accuracy is estimated without bias, and the regression cost is small. Table 13 and Appendix H.8 relax each in turn.

Worse labels. With label accuracy $\lambda = 0.5$ both methods lose ground, and the label-noise term now matters: CARVE scores 0.688, against 0.588 without the term and 0.513 for the baseline.

Biased analyst audit. If the audit overstates the analyst’s accuracy by 0.3, CARVE actually improves (0.857), because a stronger prior concentrates the budget and the evidence corrects the decoys. Understating it by 0.3 costs 12 points (0.653), still above the baseline. Under-trusting a decent analyst is thus the more costly error in this testbed.

Regression cost. Raising c_{fp} from 0.005 to 0.1 leaves CARVE between 0.78 and 0.82, while LLM-

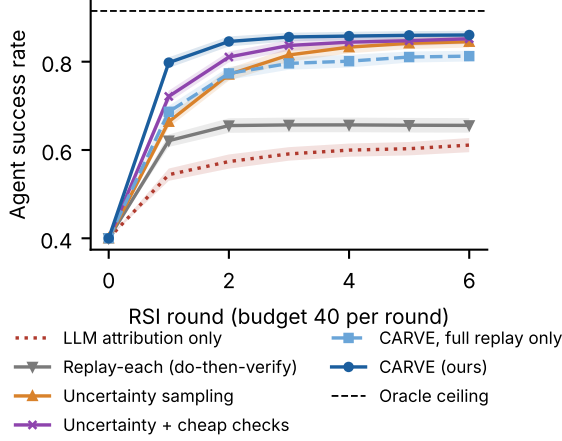


Figure 4: Agent success over six rounds of attribute-verify-patch, with a budget of 40 units per round (30 worlds, ± 1 s.e.).

only falls from 0.39 to -0.13 : the more costly a wrong change, the more verification is worth.

Joint causes. When 30% of categories need two components changed together (a single patch then repairs only a fifth as often), violating cell independence, CARVE drops from 0.787 to 0.661 at budget 50 but stays ahead of uncertainty sampling with checks (0.558; Table 16).

Correlated check errors. The one setting where CARVE loses is when the single-step judge is fooled by plausible-looking bad patches (false-positive rate 0.6 for patches with $q < 0.3$). CARVE then trusts checks that systematically pass bad patches and falls to 0.377, below every method that relies on full replays (0.58–0.61). Modelling the dependence (a quality-dependent false-positive rate in the likelihood) restores it to 0.584, on par with full replays, at a cost of 9 points when the judge is not fooled (0.684 vs. 0.776). Whether real judges behave this way is therefore the most important quantity to measure before relying on cheap checks.

Nuisance parameters and scale. Misspecifying b , λ or the patch-quality prior changes the result by at most 5 points (Table 17, Appendix H.8); misstating the check’s sensitivity and false-positive rate costs 6–14 points. On a larger graph ($I = 20$, $J = 50$; Table 15) the gap widens: at budget 50 CARVE reaches 78% of the oracle against 60% for uncertainty sampling with checks.

Inside a self-improvement loop Finally we embed each method in a loop. The agent starts at 40% success. In each round the analyst attributes 240 new failures, sampled in proportion to the current failure distribution; the method spends 40 units; accepted patches are applied, which repairs failures (true edges) or adds a regression of c_{fp} times the current failure mass (false edges); and the failure distribution is recomputed, so priorities shift towards categories that remain. Posteriors carry over between rounds.

Figure 4 shows that CARVE reaches 80% success after one round and 85% after two, close to its plateau of 86% (the oracle ceiling is 91.5%). Uncertainty sampling with checks needs four rounds to reach 84% and ends at 85%; without checks it needs six. LLM-only and Replay-each plateau at 61% and 66%: once their accepted false edges and rejected true edges are fixed in place, new attributions do not correct them.

H.5 Proofs for CARVE’s propositions

Proposition 3. Let $L_t = \ell + \sum_{u \leq t} \log \frac{P(o_u|e=1)}{P(o_u|e=0)}$ be the posterior log-odds after t observations. Under $e = 1$, the increments are i.i.d. with mean $D_1 = \text{KL}(p_1||p_0) > 0$. The rule stops at $N = \inf\{t : L_t \notin (-A, A)\}$. By Wald’s identity, $\mathbb{E}_1[L_N - \ell] = D_1 \mathbb{E}_1[N]$. Wald’s approximations give $P_1(L_N \leq -A) \approx e^{-(A+\ell)} \approx \delta e^{-\ell}$ and $P_0(L_N \geq A) \approx \delta e^{\ell}$. Neglecting overshoot and this probability (which requires $\delta e^{|\ell|} \ll 1$), $L_N = A$, hence $\mathbb{E}_1[N] = (A - \ell)/D_1$; if $\ell \geq A$ no observation is taken, which is why we assume $|\ell| < A$. Symmetrically, under $e = 0$ the drift is $-D_0$ and $\mathbb{E}_0[N] = (A + \ell)/D_0$. Taking the difference between ℓ_ρ and ℓ_{ij} and averaging over cells with $P(e=1) = \rho$ gives Δ . When the patch quality q is unknown, outcomes are exchangeable rather than i.i.d. under the marginal likelihood, and Wald’s identity does not apply directly; conditioning on q recovers the result with $D_1 = D_1(q)$, which approaches zero for a bad patch. This is the formal reason a single bad patch can stall verification and why CARVE keeps several patches per cell.

Proposition 4. By the same argument a single-step test needs $(A \mp \ell)/D_e^S$ observations at cost c_S each, and a full-replay test needs $(A \mp \ell)/D_e$ at cost c_F . The single-step test is cheaper iff $c_S/D_e^S < c_F/D_e$. Since the single-step outcome is a garbling of the full outcome, the data-processing inequality

gives $D_e^S \leq D_e$, so the critical ratio is at most one.

H.6 Simulation Details

Category frequencies are $f_i \propto 1/i$. For each category the number of true components is uniform on $\{1, 2\}$ and the decoy is a further distinct component. The audit estimate $\hat{r}$ is a binomial proportion from 20 audited attributions. The prior uses $d = 1.5$. The model’s grid has 101 points; look-ahead horizons are $\{1, 2, 4, 8\}$; the stopping threshold is 10^{-6} . Replay-each uses three replays and accepts with at least two successes. Uncertainty sampling switches to an untried patch once the current patch has three full observations. In the loop experiment, 240 attributions per round are allocated to categories by a multinomial draw with the current failure distribution. All randomness is seeded; the code reproduces every number in this paper with `python -m sim.experiments all`.

H.7 Checking Proposition 1

Table 14 compares the predicted expected number of observations with simulated sequential tests ($p_1 = 0.543$, $p_0 = 0.05$, $\delta = 0.02$, 4000 runs per row). The predictions capture the direction and approximate size of the effect of the initial log-odds; they are least accurate when $|\ell|$ is large, where overshoot and wrong-boundary exits (last column) are no longer negligible.

ℓ	e	predicted N	simulated N	wrong exit
-3	1	7.17	5.94	0.242
-3	0	1.55	2.54	0.000
-2	1	6.13	6.28	0.117
-2	0	3.28	3.73	0.001
-1	1	5.09	5.79	0.047
-1	0	5.02	5.21	0.003
+0	1	4.05	4.76	0.012
+0	0	6.75	7.28	0.009
+1	1	3.01	4.27	0.006
+1	0	8.48	8.58	0.015
+2	1	1.97	2.97	0.001
+2	0	10.22	10.23	0.068

Table 14: Proposition 3: predicted vs. simulated observations to a decision.

H.8 Additional Results

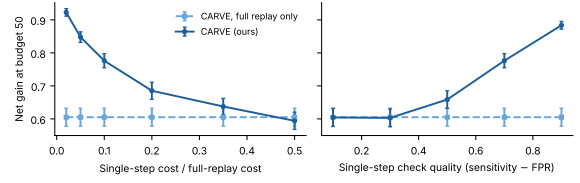


Figure 5: Effect of the single-step check’s cost (left, quality fixed at $s=0.85$, $\epsilon=0.15$) and quality (right, cost fixed at 0.1) on net gain at budget 50.

Method	50	100	300
LLM attribution only	0.231 $\pm$ 0.033	0.231 $\pm$ 0.033	0.231 $\pm$ 0.033
Replay-each (do-then-verify)	0.224 $\pm$ 0.034	0.400 $\pm$ 0.037	0.531 $\pm$ 0.032
Uncertainty sampling	0.502 $\pm$ 0.022	0.637 $\pm$ 0.020	0.822 $\pm$ 0.015
Uncertainty + cheap checks	0.603 $\pm$ 0.019	0.746 $\pm$ 0.018	0.873 $\pm$ 0.013
CARVE, full replay only	0.613 $\pm$ 0.013	0.710 $\pm$ 0.012	0.834 $\pm$ 0.010
CARVE (ours)	0.780 $\pm$ 0.016	0.860 $\pm$ 0.011	0.933 $\pm$ 0.008

Table 15: Net gain on a larger graph ($I=20$, $J=50$), 20 worlds.

Method	20	50	100	200
LLM attribution only	0.228 $\pm$ 0.028	0.228 $\pm$ 0.028	0.228 $\pm$ 0.028	0.228 $\pm$ 0.028
Replay-each	0.202 $\pm$ 0.027	0.207 $\pm$ 0.023	0.348 $\pm$ 0.033	0.362 $\pm$ 0.036
Thompson sampling	0.325 $\pm$ 0.031	0.343 $\pm$ 0.032	0.398 $\pm$ 0.032	0.484 $\pm$ 0.028
Uncertainty sampling	0.345 $\pm$ 0.036	0.465 $\pm$ 0.035	0.544 $\pm$ 0.035	0.662 $\pm$ 0.029
Uncertainty + cheap checks	0.423 $\pm$ 0.037	0.558 $\pm$ 0.035	0.615 $\pm$ 0.034	0.685 $\pm$ 0.030
CARVE (ours)	0.573 $\pm$ 0.029	0.661 $\pm$ 0.029	0.701 $\pm$ 0.027	0.745 $\pm$ 0.026

Table 16: Net gain when 30% of categories need two components changed together (30 worlds).

Model assumption (true value)	Net gain
All nuisance parameters as in the generator	0.776 $\pm$ 0.021
Spurious rate b : 0.01 (0.05)	0.811 $\pm$ 0.012
Spurious rate b : 0.15 (0.05)	0.826 $\pm$ 0.016
Label accuracy λ : 1.0 (0.8)	0.768 $\pm$ 0.022
Label accuracy λ : 0.6 (0.8)	0.794 $\pm$ 0.017
Check s/ϵ : .95/.05 (.85/.15)	0.717 $\pm$ 0.016
Check s/ϵ : .70/.30 (.85/.15)	0.636 $\pm$ 0.021
Patch prior Beta(1, 1) (Beta(2, 2))	0.811 $\pm$ 0.015
Patch prior Beta(5, 2) (Beta(2, 2))	0.777 $\pm$ 0.020

Table 17: CARVE with misspecified nuisance parameters, budget 50, 30 worlds. Several “wrong” values do slightly better than the generator’s, a reminder that the model is misspecified in other ways too.